Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №11
Реляционные и нереляционные базы данных. Хранение данных и использование CDN

1. Цель работы

Получить практические навыки проектирования и использования систем хранения данных в Python-приложениях: изучить реляционные базы данных и SQL, таблицы, первичные и внешние ключи, связи 1:N и N:M, нормализацию, транзакции и ACID, научиться подключать Python-приложение к SQLite через Repository, сравнить реляционный и нереляционный подходы (Redis), понять назначение кэширования (cache-aside, TTL, инвалидация) и CDN, а также определить, какие данные следует хранить в БД, кэшировать или отдавать через CDN.

2. Краткие теоретические сведения

Реляционная база данных представляет данные в виде таблиц. Таблица состоит из строк и столбцов, а также ограничений и ключей. Первичный ключ (PRIMARY KEY) однозначно идентифицирует запись, внешний ключ (FOREIGN KEY) устанавливает связь между таблицами. Различают три основных типа связей: один к одному (1:1), один ко многим (1:N) и многие ко многим (N:M); связь N:M реализуется через промежуточную таблицу.

Нормализация уменьшает дублирование данных и избегает аномалий при изменении информации; достаточно понимать идеи 1НФ, 2НФ и 3НФ. Основные операции SQL: SELECT, INSERT, UPDATE, DELETE; для выборки используются WHERE, ORDER BY, JOIN (INNER и LEFT), агрегатные функции COUNT, SUM, AVG, MIN, MAX и GROUP BY с HAVING.

Транзакция — последовательность операций, выполняемых как единое целое: BEGIN ... COMMIT, а при ошибке — ROLLBACK. Свойства ACID: атомарность, согласованность, изоляция и долговечность. SQLite не требует отдельного сервера и хранит всю базу в файле, поэтому удобна для лабораторных работ.

NoSQL — общее название систем хранения, не использующих классическую реляционную модель: документные (MongoDB), ключ-значение (Redis), колоночные и графовые. Redis часто используется как кэш — быстрое хранилище ключ-значение с временем жизни записей (TTL). Типовой паттерн кэширования — cache-aside: при запросе сначала проверяется кэш, при промахе данные читаются из БД и кладутся в кэш.

CDN (Content Delivery Network) — распределённая сеть серверов для доставки статического контента (изображения, CSS, JavaScript, видео) с меньшей задержкой и снижением нагрузки на исходный сервер. Кэширование управляется заголовком Cache-Control, например public, max-age=3600; актуальность ресурса проверяется по ETag (условный запрос возвращает 304 Not Modified).

3. Индивидуальное задание

Вариант 5 — предметная область «Отель». Необходимо модернизировать REST-приложение из лабораторной работы №10: данные номера (Room) должны храниться не в list/dict, а в реляционной базе данных. Нереляционная часть — Redis как кэш поиска номеров; CDN-контент — фотографии номеров и статические ресурсы (CSS, JS).

Требуется спроектировать схему БД (минимум 3 таблицы, 10 записей основной сущности, 5 связанных, 2 внешних ключа, связь 1:N и N:M), написать 15 SQL-запросов, создать минимум 2 индекса, реализовать транзакцию, перевести все endpoints лабораторной №10 на работу с БД, провести эксперимент с Redis (кэш с TTL) и раздачу статических ресурсов через CDN с исследованием HTTP-заголовков.

4. Проектирование базы данных

База гостиницы состоит из пяти таблиц: rooms (номера), guests (гости), services (дополнительные услуги), bookings (бронирования) и промежуточная таблица booking_services для связи N:M между бронированиями и услугами. Схема приведена на рисунке 11.1.

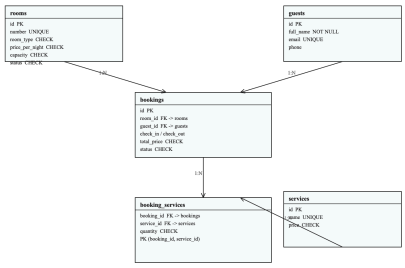


Рисунок 11.1 – ER-диаграмма базы данных гостиницы

Связи: rooms 1:N bookings (номер может иметь много бронирований), guests 1:N bookings (гость может иметь много бронирований), bookings N:M services через booking_services. Ключи: у каждой таблицы есть первичный ключ id; bookings содержит два внешних ключа (room_id, guest_id); booking_services — два внешних ключа (booking_id, service_id) и составной первичный ключ. Схема находится в третьей нормальной форме: нет повторяющихся групп (1НФ), нет частичных зависимостей от части составного ключа (2НФ), нет транзитивных зависимостей (3НФ).

Таблица	**Назначение**	**Ключи**
rooms	гостиничные номера	id PK, number UNIQUE
guests	гости	id PK
services	дополнительные услуги	id PK
bookings	бронирования	id PK, room_id FK, guest_id FK
booking_services	связь N:M бронирований и услуг	booking_id FK, service_id FK, PK(booking_id, service_id)

Файл: schema.sql

```
PRAGMA foreign_keys = ON;

DROP TABLE IF EXISTS booking_services;
DROP TABLE IF EXISTS bookings;
DROP TABLE IF EXISTS services;
DROP TABLE IF EXISTS guests;
DROP TABLE IF EXISTS rooms;

-- Основная сущность: гостиничный номер
CREATE TABLE rooms (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    number TEXT NOT NULL UNIQUE,
    room_type TEXT NOT NULL CHECK (room_type IN ('стандартный', 'люкс',
'семейный')),
    price_per_night REAL NOT NULL CHECK (price_per_night > 0),
    capacity INTEGER NOT NULL CHECK (capacity > 0),
    status TEXT NOT NULL DEFAULT 'свободен'
        CHECK (status IN ('свободен', 'занят', 'на обслуживании'))
);

CREATE TABLE guests (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    full_name TEXT NOT NULL CHECK (length(full_name) > 0),
    email TEXT NOT NULL UNIQUE,
    phone TEXT
);

CREATE TABLE services (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL UNIQUE,
    price REAL NOT NULL CHECK (price >= 0)
);

-- Бронирование: связь 1:N с номерами и гостями
CREATE TABLE bookings (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    room_id INTEGER NOT NULL REFERENCES rooms(id) ON DELETE RESTRICT,
    guest_id INTEGER NOT NULL REFERENCES guests(id) ON DELETE RESTRICT,
    check_in TEXT NOT NULL,
    check_out TEXT NOT NULL,
    total_price REAL NOT NULL CHECK (total_price >= 0),
    status TEXT NOT NULL DEFAULT 'активно'
        CHECK (status IN ('активно', 'отменено', 'завершено')),
    CHECK (check_out > check_in)
);

-- Промежуточная таблица связи N:M: бронирование и дополнительные услуги
CREATE TABLE booking_services (
    booking_id INTEGER NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
    service_id INTEGER NOT NULL REFERENCES services(id) ON DELETE RESTRICT,
    quantity INTEGER NOT NULL DEFAULT 1 CHECK (quantity > 0),
    PRIMARY KEY (booking_id, service_id)
);

-- Индекс для поиска свободных номеров и фильтра по статусу
CREATE INDEX idx_rooms_status ON rooms(status);

-- Индекс для JOIN бронирований с номерами и проверки занятости
CREATE INDEX idx_bookings_room ON bookings(room_id);

-- Индекс для выборки бронирований гостя
CREATE INDEX idx_bookings_guest ON bookings(guest_id);

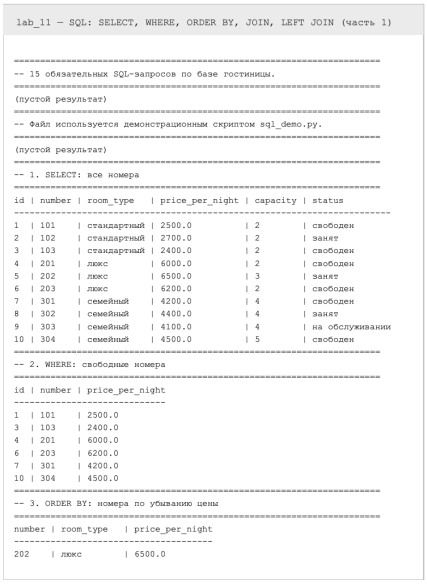
-- Индекс для сортировки и фильтрации по цене
CREATE INDEX idx_rooms_price ON rooms(price_per_night);
```

Заполнение тестовыми данными: 10 номеров, 5 гостей, 5 услуг и 6 бронирований с 8 записями связи N:M (файл seed.sql).

5. SQL-запросы

Написаны 15 обязательных запросов (файл queries.sql): SELECT всех номеров, WHERE по статусу, ORDER BY по цене, JOIN бронирований с номерами и гостями, LEFT JOIN (номера без бронирований), GROUP BY с подсчётом бронирований по типу номера, HAVING для гостей с двумя и более

бронированиями, COUNT, AVG и SUM, INSERT, UPDATE, DELETE, подзапрос (номера дороже средней цены), запрос с несколькими условиями и запрос со связью нескольких таблиц (N:M). Результаты выполнения приведены на рисунках 11.2–11.4.



```
lab_11 — SQL: SELECT, WHERE, ORDER BY, JOIN, LEFT JOIN (часть 1)


======================================================================
-- 15 обязательных SQL-запросов по базе гостиницы.
======================================================================
(пустой результат)
======================================================================
-- Файл используется демонстрационным скриптом sql_demo.py.
======================================================================
(пустой результат)
======================================================================
-- 1. SELECT: все номера
======================================================================
id | number | room_type   | price_per_night | capacity | status
----------------------------------------------------------------------
1  | 101    | стандартный | 2500.0          | 2        | свободен
2  | 102    | стандартный | 2700.0          | 2        | занят
3  | 103    | стандартный | 2400.0          | 2        | свободен
4  | 201    | люкс        | 6000.0          | 2        | свободен
5  | 202    | люкс        | 6500.0          | 3        | занят
6  | 203    | люкс        | 6200.0          | 2        | свободен
7  | 301    | семейный    | 4200.0          | 4        | свободен
8  | 302    | семейный    | 4400.0          | 4        | занят
9  | 303    | семейный    | 4100.0          | 4        | на обслуживании
10 | 304    | семейный    | 4500.0          | 5        | свободен
======================================================================
-- 2. WHERE: свободные номера
======================================================================
id | number | price_per_night
------------------------------
1  | 101    | 2500.0
3  | 103    | 2400.0
4  | 201    | 6000.0
6  | 203    | 6200.0
7  | 301    | 4200.0
10 | 304    | 4500.0
======================================================================
-- 3. ORDER BY: номера по убыванию цены
======================================================================
number | room_type   | price_per_night
---------------------------------------
202    | люкс        | 6500.0
```

Рисунок 11.2 – SQL: SELECT, WHERE, ORDER BY, JOIN, LEFT JOIN

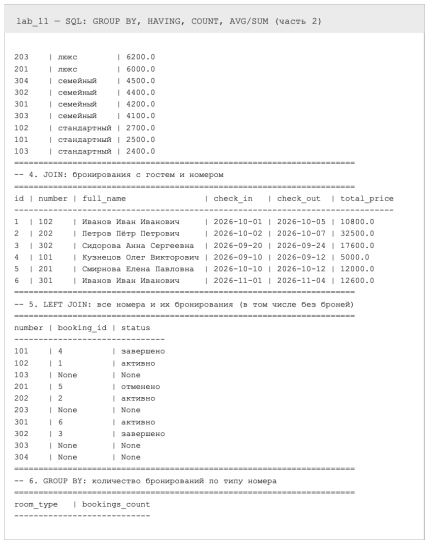


Рисунок 11.3 – SQL: GROUP BY, HAVING, COUNT, AVG, SUM



Рисунок 11.4 – SQL: INSERT, UPDATE, DELETE, подзапрос, условия, N:M

6. Индексы

В базе созданы четыре индекса: idx_rooms_status — ускоряет поиск свободных номеров (WHERE status = 'свободен') и статистику по статусам; idx_bookings_room — ускоряет соединение bookings с rooms и проверку занятости номера на период; idx_bookings_guest — ускоряет выборку бронирований конкретного гостя; idx_rooms_price — ускоряет сортировку и фильтрацию по цене. Индекс не следует создавать на каждом поле: каждый индекс увеличивает объём базы и замедляет операции записи (INSERT/ UPDATE/DELETE), поэтому индексируются только столбцы, реально используемые в WHERE, JOIN и ORDER BY.

7. Транзакции

Бизнес-операция, требующая транзакции, — создание бронирования: добавляется запись в bookings, затем позиции в booking_services (связь N:M) и номер переводится в статус «занят». Все три действия выполняются в одной транзакции: при ошибке (например, несуществующая услуга) выполняется ROLLBACK и ни одно изменение не сохраняется. Транзакция реализована в Database.transaction() (BEGIN IMMEDIATE ... COMMIT / ROLLBACK) и используется в BookingRepository.create().

Файл: database.py (фрагмент)

```
    # Транзакция: все операции выполняются целиком или откатываются
    @contextmanager
    def transaction(self) -> Iterator[sqlite3.Connection]:
        with self._lock:
            self._in_transaction = True
            try:
                self.connection.execute("BEGIN IMMEDIATE")
                yield self.connection
                self.connection.commit()
            except Exception:
                self.connection.rollback()
                raise
            finally:
                self._in_transaction = False
```

8. Интеграция с Python

Архитектура: Route → Service → Repository → Database. HTTP-обработчики не содержат SQL; репозиторий выполняет параметризованные запросы, а сервис содержит бизнес-логику и проверки. Все запросы используют параметры (?, ...) вместо конкатенации строк, что защищает от SQL-инъекций: значение подставляется как данные, а не как часть команды, поэтому ввод вроде ' OR '1'='1 не изменяет смысл запроса.

Файл: repository.py

```
class RoomRepository:
    def __init__(self, db: Database):
        self._db = db

    def list(self, filters: Dict[str, Any]) -> List[dict]:
        conditions: List[str] = []
        params: List[Any] = []

        if "room_type" in filters:
            conditions.append("room_type = ?")
            params.append(filters["room_type"])
        if "status" in filters:
            conditions.append("status = ?")
            params.append(filters["status"])
        if "min_price" in filters:
            conditions.append("price_per_night >= ?")
```

```
            params.append(filters["min_price"])
        if "max_price" in filters:
            conditions.append("price_per_night <= ?")
            params.append(filters["max_price"])

        sql = f"SELECT {ROOM_COLUMNS} FROM rooms"
        if conditions:
            sql += " WHERE " + " AND ".join(conditions)

        sort = filters.get("sort")
        if sort:
            direction = "DESC" if filters.get("order") == "desc" else "ASC"
            # имя столбца берётся из белого списка, а не из запроса пользователя
            sql += f" ORDER BY {SORT_FIELDS[sort]} {direction}"
        sql += " ORDER BY id" if not sort else ""

        return rows_to_dicts(self._db.query_all(sql, tuple(params)))

    def get(self, room_id: int) -> Optional[dict]:
        row = self._db.query_one(f"SELECT {ROOM_COLUMNS} FROM rooms WHERE id
= ?", (room_id,))
        return row_to_dict(row)

    def get_by_number(self, number: str) -> Optional[dict]:
        row = self._db.query_one(f"SELECT {ROOM_COLUMNS} FROM rooms WHERE number
= ?", (number,))
        return row_to_dict(row)

    def create(self, room: Room) -> dict:
        cursor = self._db.execute(
            "INSERT INTO rooms (number, room_type, price_per_night, capacity,
status) "
            "VALUES (?, ?, ?, ?, ?)",
            (room.number, room.room_type, room.price_per_night, room.capacity,
room.status),
        )
        return self.get(cursor.lastrowid)

    def update(self, room_id: int, data: Dict[str, Any]) -> Optional[dict]:
        if not data:
            return self.get(room_id)
        columns = ", ".join(f"{field} = ?" for field in data)
        params: Sequence[Any] = (*data.values(), room_id)
        self._db.execute(f"UPDATE rooms SET {columns} WHERE id = ?", params)
        return self.get(room_id)

    def delete(self, room_id: int) -> bool:
        cursor = self._db.execute("DELETE FROM rooms WHERE id = ?", (room_id,))
        return cursor.rowcount > 0

    def set_status(self, room_id: int, status: str) -> None:
        self._db.execute("UPDATE rooms SET status = ? WHERE id = ?", (status,
room_id))

    # Специализированный запрос варианта: свободные номера с учётом
    # вместимости и пересечения дат с активными бронированиями.
    def available(self, capacity: Optional[int] = None,
                  date_from: Optional[str] = None, date_to: Optional[str] =
None) -> List[dict]:
        conditions = ["r.status = 'свободен'"]
        params: List[Any] = []

        if capacity is not None:
            conditions.append("r.capacity >= ?")
            params.append(capacity)

        if date_from and date_to:
            conditions.append(
                "NOT EXISTS (SELECT 1 FROM bookings b "
                "WHERE b.room_id = r.id AND b.status = 'активно' "
                "AND b.check_in < ? AND b.check_out > ?)"
            )
```

```
            params.extend([date_to, date_from])

        sql = (
            f"SELECT r.id, r.number, r.room_type, r.price_per_night, r.capacity,
r.status "
            f"FROM rooms r WHERE {' AND '.join(conditions)} ORDER BY
r.price_per_night"
        )
        return rows_to_dicts(self._db.query_all(sql, tuple(params)))
```

Файл: service.py (кэш и транзакция)

```
class RoomService:
    def __init__(self, repository: RoomRepository, cache: Optional[RoomCache] =
None):
        self._repository = repository
        self._cache = cache

    def list_rooms(self, filters: Dict[str, Any]) -> List[dict]:
        return self._repository.list(filters)

    # Cache-aside для чтения одного номера
    def get_room(self, room_id: int) -> dict:
        if self._cache is not None:
            cached = self._cache.get(room_id)
            if cached is not None:
                return cached

        room = self._repository.get(room_id)
        if room is None:
            raise RoomNotFoundError(room_id)

        if self._cache is not None:
            self._cache.set(room_id, room)
        return room

    def create_room(self, payload: Any) -> dict:
        data = parse_room_payload(payload)
        if self._repository.get_by_number(data["number"]) is not None:
            raise ValidationError(f"Номер {data['number']} уже существует")
        room = Room(
            id=None,
            number=data["number"],
            room_type=data["room_type"],
            price_per_night=data["price_per_night"],
            capacity=data["capacity"],
            status=data.get("status", "свободен"),
        )
        return self._repository.create(room)

    def replace_room(self, room_id: int, payload: Any) -> dict:
        self.get_room(room_id)
        data = parse_room_payload(payload)
        room = self._repository.update(room_id, data)
        self._invalidate(room_id)
        return room

    def patch_room(self, room_id: int, payload: Any) -> dict:
        self.get_room(room_id)
        data = parse_room_payload(payload, partial=True)
        room = self._repository.update(room_id, data)
        self._invalidate(room_id)
        return room

    def delete_room(self, room_id: int) -> None:
        try:
            deleted = self._repository.delete(room_id)
        except sqlite3.IntegrityError:
            raise RoomInUseError(room_id)
        if not deleted:
            raise RoomNotFoundError(room_id)
```

```
        self._invalidate(room_id)

    def available_rooms(self, capacity: Optional[int] = None,
                        date_from: Optional[str] = None, date_to: Optional[str]
= None) -> List[dict]:
        return self._repository.available(capacity=capacity,
date_from=date_from, date_to=date_to)

    def _invalidate(self, room_id: int) -> None:
        if self._cache is not None:
            self._cache.invalidate(room_id)
```

9. NoSQL — Redis как кэш

Для варианта «Отель» выбрана система Redis: она идеально подходит для кэширования часто читаемых данных (номера) благодаря быстрым операциям key-value, поддержке TTL и инвалидации. Структура ключей: room:<id> → JSON-представление номера, срок жизни 60 секунд. Если Redis-сервер недоступен, используется fakeredis — эмуляция API Redis в памяти процесса, поэтому приложение работает в любом окружении.

Реализован паттерн cache-aside: при GET /rooms/{id} сначала проверяется кэш; при попадании данные возвращаются сразу, при промахе номер читается из SQLite и помещается в кэш. Счётчики hits и misses доступны через GET /cache/stats. После изменения (PATCH, PUT, DELETE) или бронирования номер инвалидируется из кэша, чтобы следующие запросы получили свежие данные (cache invalidation). TTL нужен, потому что данные меняются: без срока жизни кэш мог бы бесконечно отдавать устаревшие данные; по истечении 60 секунд запись автоматически удаляется и при следующем запросе данные заново читаются из БД.

Файл: cache.py

```
import json
from typing import Any, Optional, Tuple

CACHE_TTL = 60
KEY_PREFIX = "room"


# Подключение к Redis. Если Redis-сервер недоступен, используется
# fakeredis — полная эмуляция Redis API в памяти процесса.
def create_cache_client(timeout: float = 0.3) -> Tuple[Any, str]:
    try:
        import redis

        client = redis.Redis(
            host="127.0.0.1", port=6379, socket_connect_timeout=timeout,
decode_responses=True
        )
        client.ping()
        return client, "redis"
```

```
    except Exception:
        import fakeredis

        return fakeredis.FakeRedis(decode_responses=True), "fakeredis"


# Cache-aside: сначала проверяем кэш, при промахе читаем БД и кладём в кэш.
class RoomCache:
    def __init__(self, client: Any, ttl: int = CACHE_TTL):
        self._client = client
        self._ttl = ttl
        self.hits = 0
        self.misses = 0

    @staticmethod
    def key(room_id: int) -> str:
        return f"{KEY_PREFIX}:{room_id}"

    def get(self, room_id: int) -> Optional[dict]:
        raw = self._client.get(self.key(room_id))
        if raw is None:
            self.misses += 1
            return None
        self.hits += 1
        return json.loads(raw)

    def set(self, room_id: int, room: dict) -> None:
        self._client.set(self.key(room_id), json.dumps(room,
ensure_ascii=False), ex=self._ttl)

    def invalidate(self, room_id: int) -> None:
        self._client.delete(self.key(room_id))

    def ttl(self, room_id: int) -> int:
        return int(self._client.ttl(self.key(room_id)))

    def flush(self) -> None:
        for key in self._client.keys(f"{KEY_PREFIX}:*"):
            self._client.delete(key)
        self.hits = 0
        self.misses = 0
```

Результаты эксперимента cache-aside приведены на рисунке 11.5.

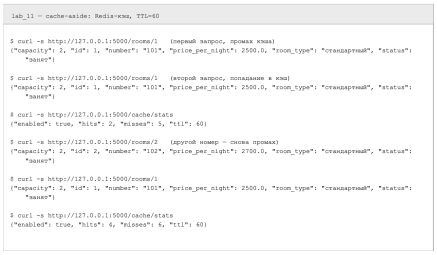


Рисунок 11.5 – Cache-aside: промахи и попадания в кэш

10. CDN

Через CDN раздаются фотографии номеров (room-101.jpg, room-201.jpg, room-301.jpg), CSS и JavaScript. Поскольку внешнее CDN недоступно, используется локальная имитация: отдельный сервер статического контента static_server.py на порту 8080, который играет роль CDN и добавляет заголовки Cache-Control и ETag, а также отвечает 304 Not Modified на условные запросы. Результаты эксперимента с HTTP-заголовками приведены на рисунке 11.6.

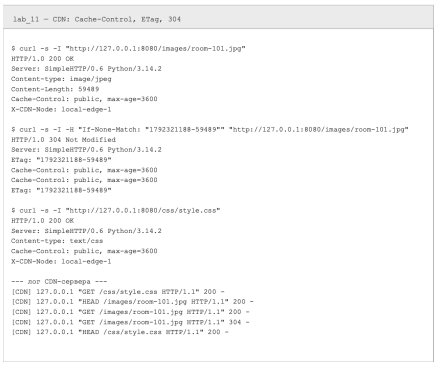


Рисунок 11.6 – CDN: заголовки HTTP, ETag, 304 Not Modified

Первый запрос к /images/room-101.jpg возвращает 200 OK с Cache-Control: public, max-age=3600 и ETag; повторный запрос с If-None-Match равным ETag возвращает 304 Not Modified без тела — клиент использует локальную копию, а трафик и нагрузка на сервер снижаются. Отличие от базы данных: в PostgreSQL хранятся изменяемые бизнес-данные (цена, статус), которые должны оставаться согласованными, тогда как фотографии и статические файлы неизменяемы в течение длительного времени и потому раздаются через CDN с большим max-age.

11. Сравнительный анализ технологий

Характеристика	PostgreSQL	MongoDB	Redis	CDN
Основное назначение	основной источник данных	гибкие документы	быстрые key-value чтения	раздача статики
Структура	жёсткая схема таблиц	гибкие документы JSON	простые ключ-значение	файлы/объекты

Характеристика	PostgreSQL	MongoDB	Redis	CDN
Хранение	на диске	на диске	в памяти	близко к пользователю
Запросы	SQL, JOIN	find/update	GET/SET/INCR	HTTP GET
Транзакции	полные ACID	ограниченные	MULTI/EXEC	нет
Кэширование	нет (источник)	нет (источник)	TTL, cache-aside	Cache-Control
Масштабирование	репликация	шардирование	кластеры	edge-серверы
Когда использовать	бизнес-данные	гибкая схема	горячие данные	изображения, CSS

Архитектурный вывод: цена номера и статус относятся к изменяемым бизнес-данным — их источником должна быть реляционная БД; часто читаемый номер можно временно кэшировать в Redis с TTL; фотографии и стили — статический контент, который отдаётся через CDN. Одна и та же информация может обслуживаться разными системами в зависимости от требований к согласованности и скорости.

12. Демонстрация работы

Тест 1. Запуск сервера Flask с подключением к SQLite и Redis-кэшу, журнал обработанных запросов (рисунок 11.7).

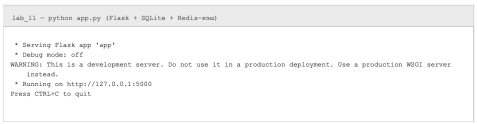


Рисунок 11.7 – Запуск сервера и журнал запросов

Тест 2. REST API с базой данных: список номеров, получение объекта, создание (201), изменение (PATCH), удаление (204), ошибки 404/400, фильтрация, свободные номера и транзакция бронирования (рисунки 11.8–11.9).

lab_11 — REST: список, CRUD (часть 1)

```
$ curl -s "http://127.0.0.1:5000/rooms"
[
 {
  "capacity": 2,
  "id": 1,
  "number": "101",
  "price_per_night": 2500.0,
  "room_type": "стандартный",
  "status": "свободен"
 },
 {
  "capacity": 2,
  "id": 2,
  "number": "102",
  "price_per_night": 2700.0,
  "room_type": "стандартный",
  "status": "занят"
 },
 {
  "capacity": 2,
  "id": 3,
  "number": "103",
  "price_per_night": 2400.0,
  "room_type": "стандартный",
  "status": "свободен"
 },
 {
  "capacity": 2,
  "id": 4,
  "number": "201",
  "price_per_night": 6000.0,
  "room_type": "люкс",
  "status": "свободен"
 },
 {
  "capacity": 3,
  "id": 5,
  "number": "202",
  "price_per_night": 6500.0,
  "room_type": "люкс",
  "status": "занят"
 },
```

Рисунок 11.8 – REST API: список, CRUD-операции

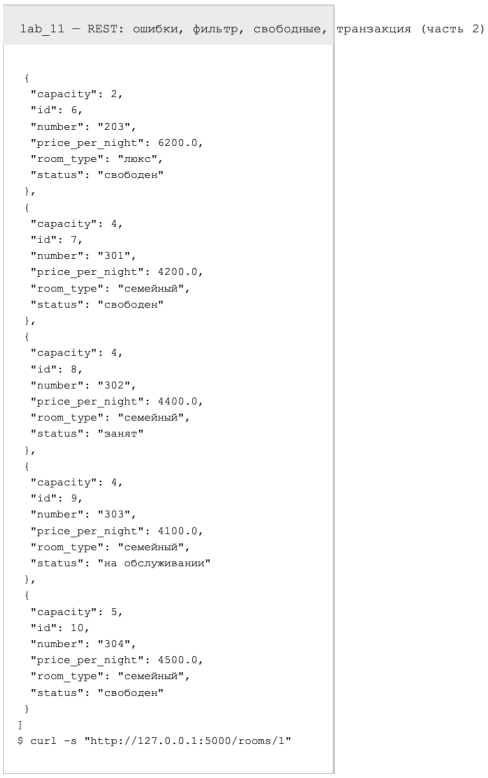


Рисунок 11.9 – REST API: ошибки, фильтр, свободные номера, транзакция

13. Тестирование

Написано 22 автоматических теста pytest: тесты репозитория (создание, чтение, изменение, удаление, специализированный запрос свободных номеров, JOIN), тесты сервиса (cache-aside, инвалидация кэша, транзакция и её откат, валидация) и тесты API (список, получение, создание, изменение, удаление,

ошибки, свободные номера, бронирование). Используется fakeredis, поэтому тесты не требуют запущенного Redis. Результат выполнения приведён на рисунке 11.10.



Рисунок 11.10 – Результаты выполнения pytest (22 passed)

14. Вывод

В ходе лабораторной работы REST-приложение гостиницы переведено с хранения в памяти на реляционную базу SQLite: спроектирована схема из пяти таблиц с первичными и внешними ключами, связями 1:N и N:M через промежуточную таблицу, написаны 15 SQL-запросов, созданы индексы и реализована транзакция создания бронирования с гарантированной целостностью данных.

Изучены и применены на практике нереляционные системы: Redis использован как кэш по паттерну cache-aside с TTL и инвалидацией, что наглядно ускоряет повторные чтения; выполнена раздача статических ресурсов через локальную имитацию CDN с исследованием заголовков Cache-Control и ETag (условные ответы 304). Проведён сравнительный анализ PostgreSQL, MongoDB, Redis и CDN и сформулирован главный вывод работы: разные типы данных требуют разных систем хранения — изменяемые бизнес-данные

хранятся в реляционной БД, горячие данные кэшируются в Redis, а статический контент раздаётся через CDN.